

Passive Risk Detection Report

devdaytest1 - Harness CD - Cluster: K8s-Cluster - Namespace: hce
Scan Identity: deploynginx-scan-v8bEFQN9Tgull8AO26O95g - Scanned 2026-05-15 07:24 UTC

HARNESS RESILIENCE TESTING - EXPERIMENTAL | Confidential



Executive Summary

This report presents the findings of a passive risk detection scan performed on the devdaytest1 pipeline deploying to Kubernetes cluster K8s-Cluster in namespace hce. The scan analyzed 1 service(s) deployed through the pipeline and identified 9 risks across the application manifests.

The severity distribution reveals 6 High-severity risks and 3 Medium-severity risks, yielding a Pipeline Risk Score of 72/1000. The majority of findings relate to missing production hardening configurations including resource limits, health probes, security contexts, and high-availability settings. All 9 risks remain in Open status requiring immediate remediation attention.

Critical coverage gaps were identified: the deployment lacks chaos engineering tests and load tests, leaving resilience assumptions unvalidated. Without resource constraints, health checks, or network policies, the nginx service is vulnerable to cascading failures, resource contention, and lateral attack movement.

Scan Metrics

Metric	Value
Service Count	1
Total Risks	9
Services Affected	1
Chaos Coverage	0%
Open Risks	9
Scan Type	pipeline

Services Found

Service	Total Risks	High	Medium	Low	RRS
nginx	9	6	3	0	72

RRS (Resilience Risk Score) = (High x 10) + (Medium x 4) + (Low x 1). Higher score indicates greater risk exposure.

Risk Details

High

Missing Resource Limits

k8s-missing-resource-limits

Container nginx in Deployment nginx has no resources.limits.cpu or resources.limits.memory defined. The container can consume unbounded node resources, causi...

Affects: deployment/nginx in namespace hce

Without memory limits, a memory leak in the nginx container can consume all available node memory, triggering the Linux OOM killer to terminate oth...

First Detected: 2026-05-15 07:24 UTC | Last Detected: 2026-05-15 07:24 UTC

kubernetes

reliability

resource-management

performance

High

Run As Root

k8s-run-as-root

Container nginx in Deployment nginx has no securityContext defined. No runAsNonRoot or runAsUser is set, meaning the container runs as root (UID 0) by default...

Affects: deployment/nginx in namespace hce

Running as root bypasses filesystem permission checks and greatly widens the impact of a container escape or remote code execution vulnerability.

First Detected: 2026-05-15 07:24 UTC | Last Detected: 2026-05-15 07:24 UTC

kubernetes

security

container-security

least-privilege

Medium

Dangerous Capabilities

k8s-dangerous-capabilities

Container nginx has no securityContext.capabilities configured. Without explicitly dropping ALL capabilities, the container inherits the default Docker/OCI c...

Affects: deployment/nginx in namespace hce

The default capability set includes capabilities like NET_RAW (allows packet crafting/sniffing), SETUID/SETGID (allows privilege escalation inside ...

First Detected: 2026-05-15 07:25 UTC | Last Detected: 2026-05-15 07:25 UTC

kubernetes

security

container-security

least-privilege

Medium

Missing HPA

k8s-missing-hpa

No HorizontalPodAutoscaler targets the nginx Deployment. The service cannot scale out under load spikes, making resource saturation a fixed-capacity ceiling.

Affects: deployment/nginx in namespace hce

With a fixed replica count of 1 and no HPA, any traffic spike beyond the single pod's capacity causes request queuing, increased latency, and event...

First Detected: 2026-05-15 07:25 UTC | Last Detected: 2026-05-15 07:25 UTC

kubernetes

availability

reliability

scaling

HighSingle Replica

k8s-single-replica

Deployment nginx in namespace hce is running with replicas: 1, making it a single point of failure with zero tolerance for pod-level disruptions.

Affects: deployment/nginx in namespace hce

If the single pod is deleted, crashes, or is evicted, the service has zero healthy endpoints and all requests fail for the entire pod restart durat...

First Detected: 2026-05-15 07:24 UTC | Last Detected: 2026-05-15 07:24 UTC

- kubernetes
- availability
- reliability
- high-availability

HighMissing Resource Requests

k8s-missing-resource-requests

Container nginx in Deployment nginx has no resources.requests.cpu or resources.requests.memory defined. The pod is classified as BestEffort QoS and will be t...

Affects: deployment/nginx in namespace hce

Without resource requests, the pod is BestEffort QoS class - the first to be evicted by the kubelet under node memory pressure, even when the pod i...

First Detected: 2026-05-15 07:24 UTC | Last Detected: 2026-05-15 07:24 UTC

- kubernetes
- reliability
- resource-management
- scheduling

HighMissing Probes

k8s-missing-probes

Container nginx in Deployment nginx has no livenessProbe or readinessProbe defined. Kubernetes cannot detect unhealthy pods and will continue routing traffic...

Affects: deployment/nginx in namespace hce

Without a readinessProbe, traffic is routed to pods that may not be ready. Without a livenessProbe, deadlocked or stuck pods are never automaticall...

First Detected: 2026-05-15 07:24 UTC | Last Detected: 2026-05-15 07:24 UTC

- kubernetes
- availability
- reliability
- health-checks

MediumMissing Network Policy

k8s-missing-network-policy

No NetworkPolicy exists that selects the nginx pods in namespace hce. All ingress and egress traffic is unrestricted, enabling lateral movement across namesp...

Affects: deployment/nginx in namespace hce

Without a NetworkPolicy, any pod in the cluster can reach the nginx service and the nginx pod can initiate connections to any other service or exte...

First Detected: 2026-05-15 07:25 UTC | Last Detected: 2026-05-15 07:25 UTC

- kubernetes
- security
- network
- zero-trust

High

Latest Image Tag

k8s-latest-image-tag

Container nginx uses image 'nginx' with no tag specified. This defaults to :latest, making deployments non-deterministic - a pod restart may pull a different...

Affects: deployment/nginx in namespace hce

Without a pinned image tag, pod restarts may pull a different image version from the registry, causing silent version drift. Rollback is impossible.

First Detected: 2026-05-15 07:25 UTC | Last Detected: 2026-05-15 07:25 UTC

kubernetes

reliability

deployment

supply-chain

Severity Distribution

Severity Level	Count	% of Total	Definition
High	6	67%	Critical risk requiring immediate remediation
Medium	3	33%	Moderate risk with potential operational impact
Low	0	0%	Minor risk with limited blast radius

Category Breakdown

Category	Count	Risks Included
Reliability	4	Missing Resource Limits, Missing Resource Requests, Missing Probes,...
Security	3	Run As Root, Dangerous Capabilities, Missing Network Policy
Availability	2	Missing HPA, Single Replica

Recommendations & Next Steps

Priority	Service	Action	Experiment Type
P1	nginx	Add resources.limits.cpu and resources.limits.m...	pod-cpu-hog / pod-memory-hog
P1	nginx	Set runAsNonRoot: true and runAsUser to non-zer...	container-kill
P1	nginx	Increase replicas to >= 2 and add PDB	pod-delete
P1	nginx	Add resources.requests.cpu and resources.reques...	node-memory-hog / node-cpu-hog
P1	nginx	Add livenessProbe and readinessProbe (httpGet p...	container-kill
P1	nginx	Pin image to specific version tag or SHA256 digest	pod-delete (image drift test)
P2	nginx	Add capabilities.drop: [ALL], add back only req...	container-kill
P2	nginx	Create HPA with CPU/memory thresholds	pod-cpu-hog (load spike)
P2	nginx	Create NetworkPolicy restricting ingress/egress	pod-network-loss

RECOMMENDED IMMEDIATE ACTION

1. Add resource limits and requests to prevent BestEffort QoS eviction and noisy-neighbour failures
2. Increase replicas to >= 2 and add health probes to eliminate single points of failure

AUTO-PARAMETERISED EXPERIMENT CONFIG

namespace: hce
blast_radius: 1 pod (single replica - full service impact)
hypothesis: Service maintains availability when single pod is terminated
duration: 60s
rollback_trigger: > 5% error rate or p99 latency > 2s

Test Coverage Gap Analysis

Service	Chaos Tests	Load Tests	Coverage %	Notes
nginx	0	0	0%	Critical - No tests configured
TOTAL	0	0	0%	Critical - Immediate action required

No chaos engineering or load testing coverage was detected for the scanned services. This represents a critical gap in resilience validation. Without active testing, the risks identified in this report remain theoretical - their actual blast radius and recovery behavior under failure conditions is unknown.

Appendix & Glossary

Report Generation Methodology

This Passive Risk Detection (PRD) report was generated by analyzing Kubernetes application manifests deployed through Harness CD pipelines. The scan engine evaluates manifests against a curated set of risk rules covering availability, reliability, security, and performance domains. Each rule uses pattern matching and structural analysis to identify configurations that deviate from production best practices and Kubernetes hardening guidelines.

Risk scores are computed using a weighted formula that assigns higher impact to High-severity findings. The Resilience Risk Score (RRS) provides a single numeric indicator for prioritization. Recommendations are auto-parameterised based on the detected risk type and target workload configuration, mapping each finding to concrete chaos experiments that validate resilience hypotheses.

Glossary

Term	Definition
RRS	Resilience Risk Score - weighted score: (High x 10) + (Medium x 4) + (Low x 1)
PDB	PodDisruptionBudget - Kubernetes resource limiting voluntary pod evictions
HPA	HorizontalPodAutoscaler - auto-scales pods based on resource metrics
MTTR	Mean Time To Recovery - average time to restore service after failure
PRD-Report	Passive Risk Detection Report - this document format
HIGH	Critical severity - immediate remediation required, significant blast radius
MEDIUM	Moderate severity - remediation recommended, contained blast radius
LOW	Low severity - best practice deviation, minimal immediate impact